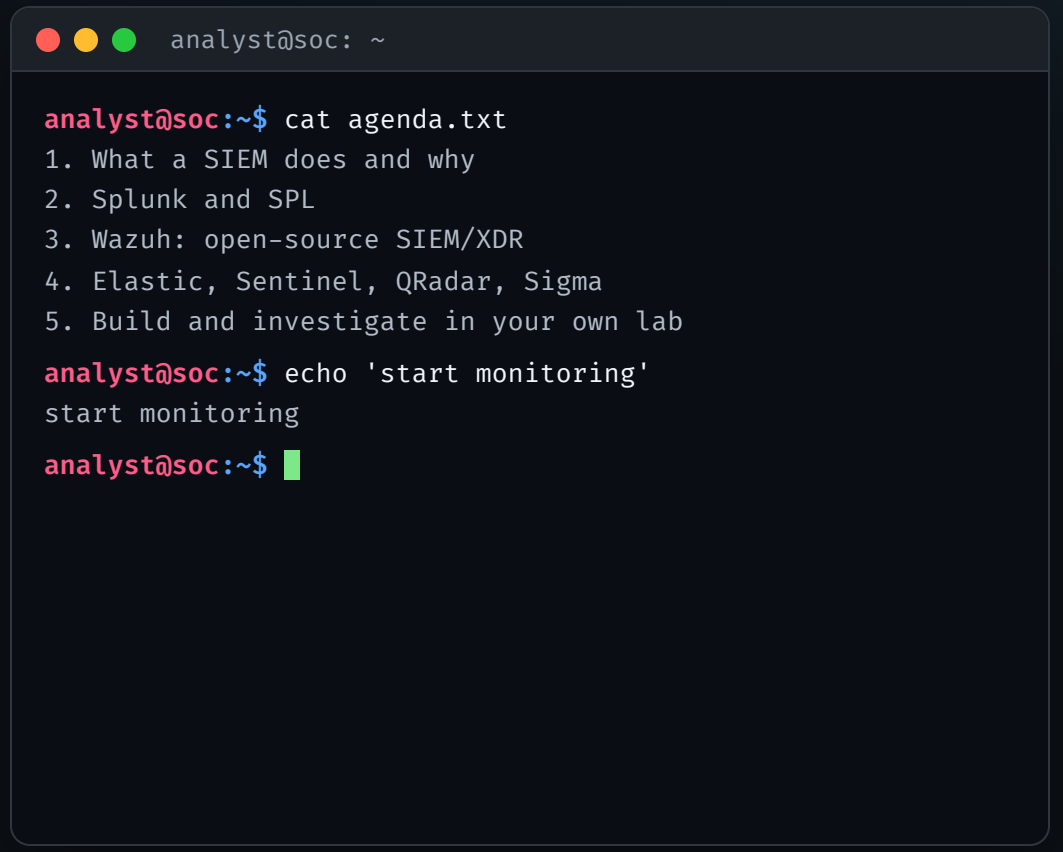
LECTURE 02 · SECURITY MONITORING

SIEM Tools

How security teams collect logs, detect attacks and respond, with Splunk, Wazuh, Elastic, Microsoft Sentinel and IBM QRadar.

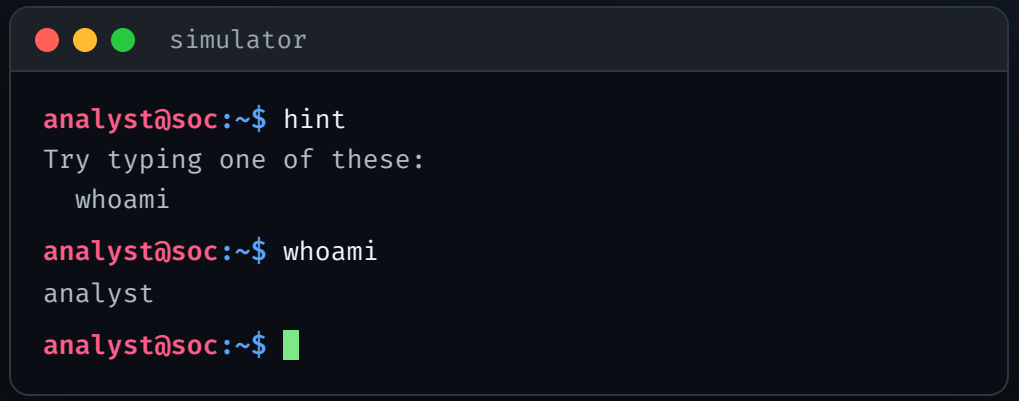
25 topics · 5 modules · query consoles on every slide



BEFORE WE START

How to use this lecture

- **Every terminal is a simulator.** Click it and type the commands or queries you see. Type `help` or `hint`.
- **Outputs are example lab data** from a host-only network (192.168.56.0/24). Your results will differ.
- **Generate test activity only against machines you own.** A SIEM lab needs no outside targets.
- Prerequisite: Lecture 01 (Linux commands). `grep`, `jq`, `sort` and `uniq` come back here.



AGENDA

Five modules

1. SIEM Fundamentals 7

What is a SIEM? · SIEM architecture · Log sources · Parsing & normalization · Correlation rules · Alert triage & SOC workflow · MITRE ATT&CK mapping

2. Splunk 6

Splunk overview · Universal Forwarder · SPL basics · SPL toolkit · Windows events in Splunk · Splunk alerts

3. Wazuh 7

Wazuh overview · Deploying Wazuh agents · Decoders & rules · Custom rules · File Integrity Monitoring · Active response · Dashboard & API

4. Elastic, Sentinel, QRadar & Sigma 5

Elastic Security · Microsoft Sentinel · IBM QRadar · Sigma: one rule, any SIEM · Comparison

5. Hands-on SOC Lab 2

Build a home SIEM lab · Investigating an alert

MODULE 1 OF 5

SIEM Fundamentals

What is a SIEM? · Log sources · Parsing & normalization · Correlation rules · Alert triage & SOC workflow · MITRE ATT&CK mapping

What is a SIEM?

Security Information and Event Management: one place that collects logs from everything, makes them searchable, and raises alerts when activity looks malicious.

`collect → normalize → correlate → alert → retain`

Collect	pull logs from servers, endpoints, network devices and cloud
Normalize	turn many formats into common field names
Correlate	link related events across sources and time
Alert	notify analysts when a rule matches
Retain	keep history for investigations and compliance

SOC TIP

Without a SIEM an analyst must log into every machine one by one. With it, one query covers the whole estate.

```
analyst@soc:~$

# the slow way: one server at a time
analyst@soc:~$ ssh web01 "grep -c 'Failed password' /var/log/auth.log"
214
analyst@soc:~$ ssh db01 "grep -c 'Failed password' /var/log/auth.log"
3
# ...and 200 more servers. In a SIEM it is one search:
analyst@soc:~$ siem-search 'event.outcome:failure' --last 24h --by host.name
host.name     count
web01           214
vpn01            57
db01              3
# (siem-search is a teaching stand-in for any SIEM's search box)
analyst@soc:~$
```

▶ **Practice** List five log sources in your own lab you would send to a SIEM, and one attack each would reveal.

SIEM FUNDAMENTALS

SIEM architecture

01
Log sources
servers · endpoints · firewall · DNS · cloud

02
Collection
agents · forwarders · syslog · APIs

03
Parse & normalize
fields · common schema · enrichment

04
Store & index
fast search · retention

05
Detect & correlate
rules · thresholds · sequences · ML

06
Respond
alerts · dashboards · cases · SOAR

Splunk
`forwarder → indexer → search head`

Wazuh
`agent → server → indexer → dashboard`

Elastic
`Elastic Agent → Elasticsearch → Kibana`

Sentinel
`connectors → Log Analytics → analytics rules`

Log sources

A SIEM is only as good as the data it receives. These sources give the most security value per byte.

`endpoints · identity · network · applications · cloud`

Linux	/var/log/auth.log, journald, auditd
Windows	Security log: 4624 logon, 4625 failed, 4688 process, 4720 user created
Sysmon	Event 1 process create, 3 network connect, 11 file create
Network	firewall, DNS, proxy, IDS (Suricata/Zeek)
Cloud / SaaS	AWS CloudTrail, Entra ID sign-ins, M365 audit

SOC TIP

Start with authentication logs everywhere: most intrusions involve logging in somewhere.

```
analyst@soc:~$
analyst@soc:~$ sudo tail -1 /var/log/auth.log
Oct 24 08:51:02 web01 sshd[2210]: Failed password for root from 192.168.56.23 p
ort 50412 ssh2
analyst@soc:~$ tail -1 /var/log/apache2/access.log
192.168.56.23 - - [24/Oct/2026:08:55:10 +0000] "GET /admin HTTP/1.1" 403 492
analyst@soc:~$ jq -r '.EventID, .TargetUserName, .IpAddress' event-4625.json
4625
administrator
192.168.56.23
analyst@soc:~$
```

▶ **Practice** For each Windows event ID on this slide, write one sentence on what an attacker action would create it.

Parsing & normalization

Every product writes logs differently. The SIEM extracts fields and maps them to a common schema so one query works across all sources.

`raw text → fields → common schema → enrichment`

Field extraction	regex or decoders pull out user, IP, action
Schemas	Splunk CIM, Elastic ECS, Sentinel ASIM, OCSF
Time	normalize timestamps to UTC or correlation breaks
Enrichment	add GeoIP, asset owner, threat-intel matches

SOC TIP

If two sources call the same thing 'src', 'SourceIP' and 'client_ip', your rule silently misses two of them. Normalization fixes that.

```
analyst@soc:~$
analyst@soc:~$ head -1 raw/auth.log
Oct 24 08:51:02 web01 sshd[2210]: Failed password for root from 192.168.56.23 p
ort 50412 ssh2
analyst@soc:~$ head -1 normalized/auth.json | jq
{
  "@timestamp": "2026-10-24T08:51:02Z",
  "host.name": "web01",
  "event.category": "authentication",
  "event.outcome": "failure",
  "user.name": "root",
  "source.ip": "192.168.56.23",
  "source.geo.country_iso_code": "private"
}
analyst@soc:~$
```

▶ **Practice** Take three different log lines from your lab and write the ECS field names you would map them to.

Correlation rules

A correlation rule links several events into one meaningful alert, so analysts see incidents instead of noise.

`threshold · sequence · first-seen · indicator match`

`Threshold`	N events within T minutes (e.g. 10 failed logins in 5 min)
`Sequence`	A then B (many failures, then a success)
`First-seen / rarity`	new admin account, new country, new process
`Indicator match`	IP, domain or hash on a threat-intel list

> **SOC TIP**
>
> Correlation turns thousands of raw events into the one alert worth a human's time.

analyst@soc:~$

```
analyst@soc:~$ cat rules/bruteforce_then_success.yml
name: Brute force followed by successful login
severity: high
group_by: [source.ip, user.name]
window: 10m
sequence:
  - match: {event.category: authentication, event.outcome: failure}
    count: '⩾ 10'
  - match: {event.category: authentication, event.outcome: success}
mitre: [T1110, T1078]
# generic pseudo-rule: every SIEM writes this in its own language
analyst@soc:~$
```

▶ **Practice** Write in plain English one threshold rule and one sequence rule for your lab web server.

Alert triage & SOC workflow

Every alert follows the same path. Good notes at each step make the next analyst faster.

`triage → investigate → contain → close`

Triage	real or noise? how severe? which asset?
Investigate	scope it: which hosts, users, time range
Contain	block IP, disable account, isolate host
Close	true positive, false positive or benign; document it

> **SOC TIP**
> A false positive is still useful: tune the rule so it doesn't fire again for the same benign reason.

```
analyst@soc:~$
analyst@soc:~$ cat cases/case-0142.md
# Case 0142: SSH brute force on web01
Alert:    5712 sshd brute force (level 10)
Source:   192.168.56.23 (lab attacker VM)
Scope:    29 failures, 0 successes, web01 only
Action:   IP blocked by active response for 10 min
Verdict:  True positive (authorized lab test)
Tuning:   none needed
analyst@soc:~$
```

▶ **Practice** Write a case note like this for one alert you generate in your lab this week.

MITRE ATT&CK mapping

ATT&CK is a public catalogue of attacker techniques. Tagging rules with technique IDs shows what you detect and where the gaps are.

`tactic (why) → technique (how) → your detection`

T1110	Brute Force
T1078	Valid Accounts
T1059	Command and Scripting Interpreter
T1136	Create Account
T1053	Scheduled Task/Job

SOC TIP

Build an ATT&CK coverage map from your rule tags; uncovered techniques become your detection backlog.

```
analyst@soc:~$
analyst@soc:~$ grep -H 'attack.t' rules/*.yml
rules/ssh_bruteforce.yml:    - attack.t1110
rules/new_local_admin.yml:   - attack.t1136
rules/cron_modified.yml:     - attack.t1053
analyst@soc:~$ grep -h 'attack.t' rules/*.yml | sort | uniq -c
      1 - attack.t1053
      1 - attack.t1110
      1 - attack.t1136
analyst@soc:~$
```

▶ **Practice** Open attack.mitre.org, pick one technique, and describe which log source would detect it.

MODULE 2 OF 5

Splunk

Splunk overview · Universal Forwarder · SPL basics · SPL toolkit · Windows events in Splunk · Splunk alerts

07/25 SPLUNK

Splunk overview

A commercial data platform widely used as a SIEM. Data is indexed on arrival and searched with SPL, the Search Processing Language.

`forwarder → indexer → search head`

`Universal Forwarder`	lightweight agent that ships logs
`Indexer`	parses, indexes and stores events
`Search head`	web UI and search engine
`Ports`	8000 web · 8089 management · 9997 forwarder input
`Enterprise Security`	premium SIEM app: notables, risk-based alerting

SOC TIP

Splunk offers a trial and a limited free license for learning; check splunk.com for current terms before building a lab.

```
analyst@splunk01:~$
analyst@splunk01:~$ sudo /opt/splunk/bin/splunk status
splunkd is running (PID: 1874).
splunk helpers are running (PIDs: 1880 1902).
analyst@splunk01:~$ sudo /opt/splunk/bin/splunk enable listen 9997
Listening for Splunk data on TCP port 9997.
analyst@splunk01:~$ sudo ss -tln | grep -E ':(8000|8089|9997)'
LISTEN 0  128  0.0.0.0:8000  0.0.0.0:*
LISTEN 0  128  0.0.0.0:8089  0.0.0.0:*
LISTEN 0  128  0.0.0.0:9997  0.0.0.0:*
analyst@splunk01:~$
```

▶ **Practice** Install Splunk on a lab VM, open port 9997 for receiving, and log in at http://<vm-ip>:8000.

08/25 SPLUNK

Universal Forwarder

Installed on each source host, the forwarder watches files and sends new lines to the indexer.

`splunk add forward-server · splunk add monitor`

`add forward-server`	where to send data (indexer:9997)
`add monitor`	which file or folder to watch
`-index`	destination index (create it on the indexer first)
`-sourcetype`	tells Splunk how to parse it
`outputs.conf / inputs.conf`	the config files these commands write

SOC TIP

Use a dedicated index per data type (linux, wineventlog, web) so searches and retention stay manageable.

```
analyst@web01:~$

# on web01
analyst@web01:~$ sudo /opt/splunkforwarder/bin/splunk add forward-server 192.168.56.20:9997
Added forwarding to: 192.168.56.20:9997.
analyst@web01:~$ sudo /opt/splunkforwarder/bin/splunk add monitor /var/log/auth.log -index linux -sourcetype linux_secure
Added monitor of '/var/log/auth.log'.
analyst@web01:~$ sudo /opt/splunkforwarder/bin/splunk list forward-server
Active forwards:
        192.168.56.20:9997
Configured but inactive forwards:
        None
analyst@web01:~$
```

▶ **Practice** Forward /var/log/auth.log from a second VM and confirm events arrive with index=linux.

09/25 SPLUNK

SPL basics

SPL searches start by choosing data, then pipe results through commands, like a Linux pipeline.

```
index=… sourcetype=… "text" | command | command
```

`index=` / `sourcetype=`	where to look (always set them, it's faster)
`|`	pipe results into the next command
`rex`	extract a field with a regex
`stats count by`	group and count
`sort -count`	sort descending

SOC TIP

Same idea as Lecture 01's grep | awk | sort | uniq -c, but across every host at once.

splunk › search

```
splunk › search index=linux sourcetype=linux_secure "Failed password" | head 2
Oct 24 08:51:05 web01 sshd[2210]: Failed password for root from 192.168.56.23 p
ort 50418 ssh2
Oct 24 08:51:02 web01 sshd[2210]: Failed password for root from 192.168.56.23 p
ort 50412 ssh2
splunk › search index=linux "Failed password" | rex "from (?<src_ip>\d+\.\d+
\.\d+\.\d+)" | stats count by src_ip | sort -count
src_ip            count
192.168.56.23        29
192.168.56.41         6
192.168.56.7          2
splunk › search
```

▶ **Practice** Write a search that counts failed logins per user instead of per source IP.

10/25 SPLUNK

SPL toolkit

A handful of commands cover most day-to-day hunting and reporting.

`timechart · table · top · where · eval · dedup`

`timechart`	counts over time (spot spikes)
`table`	choose columns to display
`top`	most common values
`where`	filter on calculated values
`eval`	create or transform fields
`dedup`	keep one event per value

SOC TIP

timechart is the fastest way to see when an attack started and stopped.

splunk › search

```
splunk › search index=linux "Failed password" | timechart span=10m count
_time               count
2026-10-24 08:40        0
2026-10-24 08:50       31
2026-10-24 09:00        6
splunk › search index=linux "Accepted" | rex "for (?<user>\S+) from (?<src_ip>
\S+)" | table _time user src_ip
_time                 user     src_ip
2026-10-24 09:14:10   kali     192.168.56.1
splunk › search index=linux "Failed password" | top limit=2 user
user      count  percent
root         31  83.8
admin         6  16.2
splunk › search
```

▶ **Practice** Build a timechart of successful logins per user over the last 7 days.

11/25 SPLUNK

Windows events in Splunk

Windows Security events are the core of most enterprise detections. Know the key IDs by heart.

```
index=wineventlog EventCode=…
```

4624 / 4625	successful / failed logon
4672	special privileges assigned (admin logon)
4688	new process created
4720	user account created
4732	member added to a security-enabled local group

SOC TIP

A new account (4720) quickly added to Administrators (4732) is a classic persistence pattern worth an alert.

splunk › search

```
splunk › search index=wineventlog EventCode=4625 | stats count by Account_Name, Source_Network_Address | where count > 10
Account_Name   Source_Network_Address  count
administrator  192.168.56.23             18
splunk › search index=wineventlog (EventCode=4720 OR EventCode=4732) | table _time EventCode user src_user
_time                EventCode  user        src_user
2026-10-24 10:02:11  4720       svc_backup  labadmin
2026-10-24 10:02:40  4732       svc_backup  labadmin
splunk › search
```

▶ **Practice** Search for 4672 events and list which accounts logged on with admin rights today.

12/25 SPLUNK

Splunk alerts

Any search can be saved as an alert that runs on a schedule and acts when results appear.

`search → schedule → trigger condition → action`

`Schedule`	cron, e.g. */5 * * * * (every 5 minutes)
`Time range`	last 15 minutes, overlapping the schedule
`Trigger`	number of results > 0
`Actions`	email, webhook, notable event (ES)
`Throttle`	suppress repeats per src_ip

SOC TIP

This search is the sequence rule from Module 1, written in SPL.

splunk › search

```
splunk › search index=linux sourcetype=linux_secure ("Failed password" OR "Acce
pted password") | rex "(?<action>Failed|Accepted) password for (invalid user )?
(?<user>\S+) from (?<src_ip>\S+)" | stats count(eval(action="Failed")) as failu
res count(eval(action="Accepted")) as successes by src_ip user | where failures
⩾10 AND successes⩾1
src_ip          user      failures  successes
192.168.56.23   analyst         29          1
# save as alert: every 5 min, last 15 min, trigger if results > 0
splunk › search
```

▶ **Practice** Save this search as an alert in your lab and trigger it from your attacker VM.

MODULE 3 OF 5

Wazuh

Wazuh overview · Deploying Wazuh agents · Decoders & rules · Custom rules · File Integrity

Monitoring · Active response · Dashboard & API

Wazuh overview

A free, open-source security platform (SIEM + XDR): endpoint agents, rule-based detection, file integrity monitoring, vulnerability detection and compliance.

`agent → server (manager) → indexer → dashboard`

Agent	runs on endpoints, collects logs and checks
Server	decoders + rules analyse events, raise alerts
Indexer	stores and searches alerts (OpenSearch-based)
Dashboard	web UI on 443
Ports	1514 agent events · 1515 enrollment · 55000 API

SOC TIP

Wazuh is the easiest full SIEM to run at home: one VM, one install script, all features.

```
analyst@wazuh:~$
analyst@wazuh:~$ curl -sO https://packages.wazuh.com/4.x/wazuh-install.sh
analyst@wazuh:~$ sudo bash ./wazuh-install.sh -a
INFO: Starting Wazuh installation assistant.
INFO: ── Wazuh indexer ──
INFO: ── Wazuh server ──
INFO: ── Wazuh dashboard ──
INFO: You can access the web interface https://<wazuh-dashboard-ip>:443
    User: admin
    Password: <generated, also saved in wazuh-install-files.tar>
analyst@wazuh:~$ systemctl is-active wazuh-manager wazuh-indexer wazuh-dashboar
d
active
active
active
analyst@wazuh:~$
```

▶ **Practice** Run the all-in-one install on a lab VM (the docs recommend 4 vCPU / 8 GB RAM) and log into the dashboard.

Deploying Wazuh agents

Each monitored host runs an agent that enrolls with the manager and streams events over an encrypted channel.

```
WAZUH_MANAGER=… apt-get install wazuh-agent
```

`WAZUH_MANAGER`	manager IP, set at install time
`WAZUH_AGENT_NAME`	how the host appears in the dashboard
`agent_control -l`	list agents and their status (on the manager)
`ossec.conf`	agent config: /var/ossec/etc/ossec.conf

SOC TIP

An agent showing 'Disconnected' is itself a signal: on a server it may mean someone stopped it.

analyst@lab:~$

```
# on web01 (after adding the Wazuh apt repository)
analyst@lab:~$ sudo WAZUH_MANAGER='192.168.56.20' WAZUH_AGENT_NAME='web01' apt-get install -y wazuh-agent
Setting up wazuh-agent ...
analyst@lab:~$ sudo systemctl enable --now wazuh-agent
# on the manager
analyst@lab:~$ sudo /var/ossec/bin/agent_control -l
Wazuh agent_control. List of available agents:
   ID: 000, Name: wazuh (server), IP: 127.0.0.1, Active/Local
   ID: 001, Name: web01, IP: any, Active
analyst@lab:~$
```

▶ **Practice** Enroll two agents (Linux and, if you can, Windows) and confirm both show as Active.

Decoders & rules

Decoders extract fields from a log line; rules match those fields and assign an ID and a level from 0 to 15.

`/var/ossec/bin/wazuh-logtest`

`5716`	sshd: authentication failed (level 5)
`5720`	sshd: multiple authentication failures (level 10)
`5710 / 5712`	non-existent user / brute force with non-existent users
`5715`	sshd: authentication success
`wazuh-logtest`	paste a log line, see decoding and the matching rule

SOC TIP

Always test a new log format with wazuh-logtest before relying on it in production.

```
analyst@wazuh:~$
analyst@wazuh:~$ sudo /var/ossec/bin/wazuh-logtest
Type one log per line
Oct 24 08:51:02 web01 sshd[2210]: Failed password for root from 192.168.56.23 p
ort 50412 ssh2
**Phase 2: Completed decoding.
        name: 'sshd'
        srcip: '192.168.56.23'
        srcuser: 'root'
**Phase 3: Completed filtering (rules).
        id: '5716'
        level: '5'
        description: 'sshd: authentication failed.'
**Alert to be generated.
analyst@wazuh:~$
```

▶ **Practice** Paste a successful SSH login line into wazuh-logtest. Which rule ID matches?

Custom rules

Add your own detections in local_rules.xml. They build on built-in rules and survive upgrades.

`/var/ossec/etc/rules/local_rules.xml`

`id`	use 100000–120000 for custom rules
`level`	0 (ignore) to 15 (critical)
`if_sid`	only evaluate when a parent rule fired
`frequency` / `timeframe`	threshold correlation
`mitre`	tag with an ATT&CK technique

SOC TIP

This rule escalates a successful SSH login from outside the lab subnet to level 12.

analyst@wazuh:~$

```
analyst@wazuh:~$ sudo cat /var/ossec/etc/rules/local_rules.xml
<group name="local,sshd,">
  <rule id="100100" level="12">
    <if_sid>5715</if_sid>
    <srcip>!192.168.56.0/24</srcip>
    <description>SSH login from outside the lab subnet</description>
    <mitre><id>T1078</id></mitre>
  </rule>
</group>
analyst@wazuh:~$ sudo systemctl restart wazuh-manager
analyst@wazuh:~$
```

▶ **Practice** Write a rule that raises level 12 when root logs in successfully over SSH.

File Integrity Monitoring

Syscheck hashes watched files and alerts when they are added, changed or deleted.

`<syscheck><directories …>`

`realtime`	alert immediately instead of on a schedule
`report_changes`	include a diff of text files
`550`	integrity checksum changed
`554`	file added to the system
`553`	file deleted

SOC TIP

Watch /etc, web roots and startup locations: attackers change them to persist.

```
analyst@web01:~$
analyst@web01:~$ sudo grep -A2 '<syscheck>' /var/ossec/etc/ossec.conf
<syscheck>
  <directories check_all="yes" realtime="yes" report_changes="yes">/etc,/var/ww
w/html</directories>
  <frequency>43200</frequency>
analyst@web01:~$ echo '10.0.0.66 bank.example' | sudo tee -a /etc/hosts
10.0.0.66 bank.example
# on the manager
analyst@web01:~$ sudo tail -1 /var/ossec/logs/alerts/alerts.json | jq -r '.rul
e.id, .rule.description, .syscheck.path'
550
Integrity checksum changed.
/etc/hosts
analyst@web01:~$
```

▶ **Practice** Enable realtime FIM on /var/www/html, edit index.html, and find the alert with its diff.

Active response

Wazuh can run a script automatically when a rule fires, for example blocking the source IP for a few minutes.

`<active-response>` in the manager's ossec.conf

`command`	firewall-drop, host-deny, disable-account …
`location`	local (the agent that saw it) or all
`rules_id`	which rules trigger it
`timeout`	seconds before the action is undone
`white_list`	IPs never to block (your admin IP!)

SOC TIP

Automation cuts response time, but a bad rule can lock you out. Whitelist yourself and start with short timeouts.

```
analyst@wazuh:~$
analyst@wazuh:~$ sudo grep -A5 '<active-response>' /var/ossec/etc/ossec.conf
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>5712,5720</rules_id>
  <timeout>600</timeout>
# on web01, after a lab brute-force test
analyst@wazuh:~$ sudo iptables -L INPUT -n | grep 192.168.56.23
DROP       all  --  192.168.56.23        0.0.0.0/0
analyst@wazuh:~$
```

▶ **Practice** Enable firewall-drop for rule 5720, trigger it from your attacker VM, and watch the block expire.

Dashboard & API

The dashboard groups features into modules; the REST API lets scripts query and manage everything.

`https://<manager>:55000`

`Threat Hunting`	search all alerts
`FIM`	file changes per agent
`Vulnerability Detection`	CVEs in installed packages
`SCA`	configuration checks against CIS benchmarks
`MITRE ATT&CK`	alerts mapped to techniques

> **SOC TIP**
> Never hard-code API credentials in scripts you share. Read them from environment variables.

analyst@soc:~$

```
analyst@soc:~$ TOKEN=$(curl -sk -u "$WAZUH_USER:$WAZUH_PASS" -X POST 'https://192.168.56.20:55000/security/user/authenticate?raw=true')
analyst@soc:~$ curl -sk -H "Authorization: Bearer $TOKEN" 'https://192.168.56.20:55000/agents?select=name,status' | jq -c '.data.affected_items[]'
{"name":"wazuh","status":"active","id":"000"}
{"name":"web01","status":"active","id":"001"}
analyst@soc:~$
```

▶ **Practice** Open the SCA module for one agent. Pick a failed CIS check and fix it on the VM.

MODULE 4 OF 5

Elastic, Sentinel, QRadar & Sigma

Elastic Security · Microsoft Sentinel · IBM QRadar · Sigma: one rule, any SIEM

Elastic Security

Built on the Elastic Stack: Elasticsearch stores data, Kibana is the UI, and Elastic Agent or Beats collect logs. Data follows ECS.

`KQL · EQL · ES|QL`

`KQL`	quick filters in Kibana
`ES|QL`	piped query language for aggregation
`EQL`	event sequences across time
`Elastic Agent + Fleet`	centrally managed collection
`Detection rules`	prebuilt rules mapped to ATT&CK

SOC TIP

EQL sequences express 'A then B by the same user' directly, which is ideal for multi-step attacks.

kibana ›

```
kibana › event.category:authentication and event.outcome:failure and source.ip:
192.168.56.23
29 hits
kibana › FROM logs-* | WHERE event.outcome == "failure" | STATS attempts = COUN
T(*) BY source.ip | SORT attempts DESC | LIMIT 3
attempts  source.ip
      29  192.168.56.23
       6  192.168.56.41
       2  192.168.56.7
kibana › sequence by user.name with maxspan=5m [authentication where event.outc
ome == "failure"] with runs=5 [authentication where event.outcome == "success"]
1 sequence found · user.name: analyst
kibana ›
```

▶ **Practice** Rewrite the Splunk brute-force search from Module 2 in ES|QL.

Microsoft Sentinel

A cloud-native SIEM on Azure. Data lands in a Log Analytics workspace and is queried with KQL (Kusto Query Language).

```
Table | where … | summarize … by …
```

`Data connectors`	Entra ID, M365, Defender, Syslog, CEF
`Analytics rules`	scheduled KQL that creates incidents
`Workbooks`	dashboards
`Playbooks`	Logic Apps automation (SOAR)
`Hunting`	saved KQL hunting queries

SOC TIP

Note: this KQL (Kusto) is a different language from Elastic's KQL (Kibana) despite the same acronym.

sentinel › KQL

```
sentinel › KQL SecurityEvent | where TimeGenerated > ago(1h) and EventID == 462
5 | summarize Attempts = count() by IpAddress, TargetAccount | sort by Attempts
desc | take 2
IpAddress       TargetAccount         Attempts
192.168.56.23   LAB\administrator         18
192.168.56.41   LAB\backup                4

sentinel › KQL SigninLogs | where ResultType ≠ "0" | summarize Failures = coun
t() by UserPrincipalName, Location | take 2
UserPrincipalName        Location  Failures
student1@lab.example     IQ              7
student2@lab.example     DE              3

sentinel › KQL
```

▶ **Practice** Use Microsoft's free KQL training data (the Log Analytics demo workspace) to run a summarize query.

IBM QRadar

A commercial SIEM that groups related events into 'offenses'. Searches use AQL, an SQL-like language.

```
SELECT … FROM events WHERE … LAST n HOURS
```

`Log sources / DSMs`	parsers for each product
`Offenses`	correlated incidents with a magnitude score
`Flows`	network flow records alongside events
`AQL`	SQL-style searching
`Reference sets`	watchlists of IPs, users, hashes

SOC TIP

If you know SQL, AQL feels familiar: SELECT, WHERE, GROUP BY, ORDER BY, plus a time clause.

qradar › AQL

```
qradar › AQL SELECT sourceip, username, COUNT(*) AS attempts FROM events WHERE
QIDNAME(qid) ILIKE '%failed%' GROUP BY sourceip, username ORDER BY attempts DES
C LAST 1 HOURS
sourceip        username       attempts
192.168.56.23   root                 29
192.168.56.41   admin                 6
qradar › AQL SELECT DATEFORMAT(starttime,'HH:mm') AS t, sourceip, destinationpo
rt FROM events WHERE sourceip = '192.168.56.23' LAST 10 MINUTES
t      sourceip        destinationport
08:51  192.168.56.23   22
08:55  192.168.56.23   80
qradar › AQL
```

▶ **Practice** Translate the AQL query above into SPL and into Sentinel KQL.

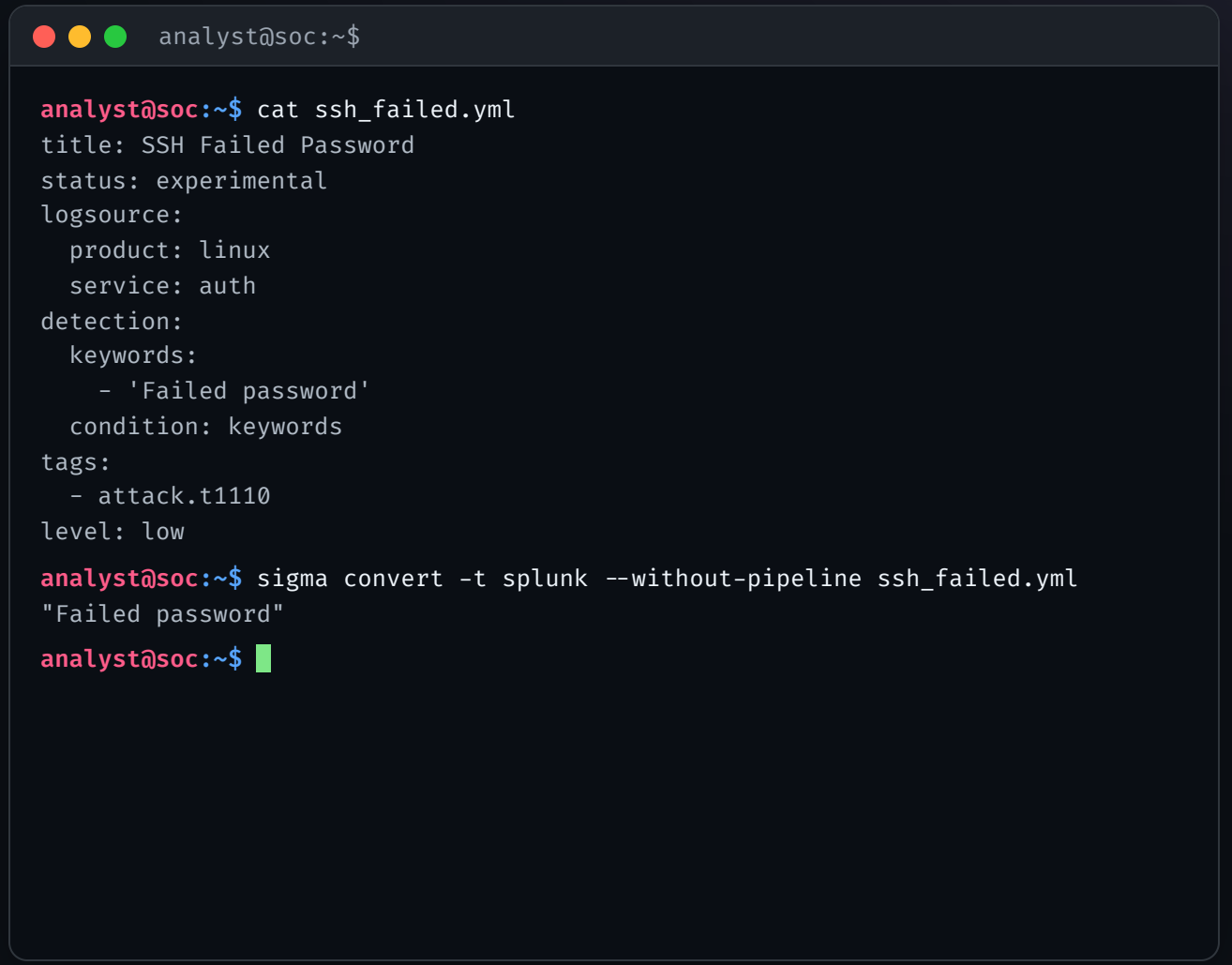
Sigma: one rule, any SIEM

Sigma is an open, YAML-based format for detection rules. Write a rule once and convert it to SPL, KQL, ES|QL and more.

```
sigma convert -t <backend> rule.yml
```

`logsource`	what data the rule needs
`detection`	selections and a condition
`tags`	ATT&CK mapping
`sigma-cli + backends`	pip install sigma-cli pySigma-backend-splunk
`SigmaHQ`	thousands of community rules on GitHub

> **SOC TIP**
> Sigma keeps detections portable when your organisation changes SIEM vendor.



▶ **Practice** Browse github.com/SigmaHQ/sigma, pick one Linux rule, and convert it for your SIEM.

ELASTIC, SENTINEL, QRADAR & SIGMA

Choosing a SIEM

TOOL	MODEL	QUERY LANGUAGE	STRONG AT
Splunk	Commercial	`SPL`	Large SOCs, flexible search, big app ecosystem
Wazuh	Free, open source	`Dashboard search + XML rules`	Endpoints, FIM, compliance, home labs
Elastic Security	Open + paid tiers	`KQL, EQL, ES|QL`	Search at scale, prebuilt detection rules
Microsoft Sentinel	Cloud (Azure)	`KQL (Kusto)`	Microsoft 365 and Azure environments
IBM QRadar	Commercial	`AQL`	Offense-based correlation, flows
Graylog	Open + paid tiers	`Graylog search syntax`	Log management on a budget
Security Onion	Free, open source	`SOC UI / hunting queries`	Network security monitoring (Suricata, Zeek)

The concepts transfer: learn one query language well and the others take days, not months.

MODULE 5 OF 5

Hands-on SOC Lab

Build a home SIEM lab · Investigating an alert

Build a home SIEM lab

Three VMs on a host-only network are enough to practise the full detect-and-respond loop safely.

`SIEM VM · target VM · attacker VM`

`192.168.56.20`	Wazuh all-in-one (or Splunk)
`192.168.56.10`	Ubuntu web server with agent / forwarder
`192.168.56.23`	Kali, used to generate test activity
`Snapshots`	take one before every experiment

SOC TIP

Only generate test activity against machines you own. These events are harmless but real.

```
kali@kali:~$

# from Kali: write a harmless marker event into syslog on the target
kali@kali:~$ ssh analyst@192.168.56.10 "logger -p auth.warning 'SIEM-LAB marker event'"
# from Kali: 10 failed logins against your own lab server
kali@kali:~$ for i in $(seq 10); do ssh -o BatchMode=yes nosuchuser@192.168.56.10 true; done 2>/dev/null
# now search for both in your SIEM
kali@kali:~$ sudo grep -c 'nosuchuser' /var/log/auth.log
10
kali@kali:~$
```

▶ **Practice** Build the lab, generate both events, and find them in your SIEM within 5 minutes.

Investigating an alert

Start from the alert, pivot on its key fields (IP, user, host), and build the timeline.

`alert → pivot → timeline → verdict`

Pivot on IP	what else did this source do?
Pivot on user	where else did this account log in?
Timeline	first seen, last seen, sequence of events
Verdict	document and tune

SOC TIP

Lecture 01 tools still matter: jq, grep, sort and uniq work on SIEM exports too.

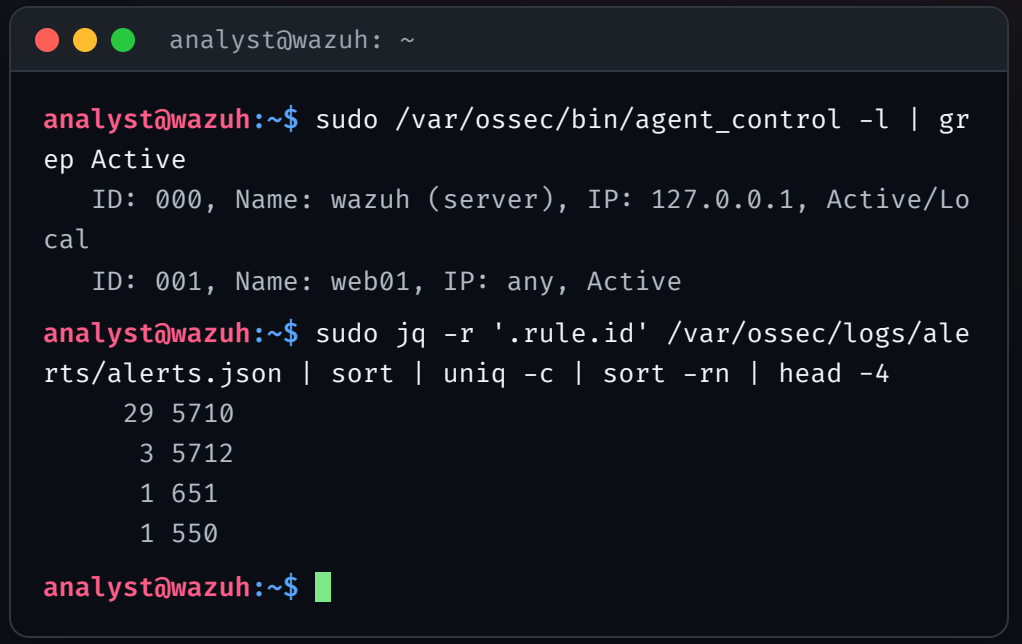
```
analyst@wazuh:~$
analyst@wazuh:~$ sudo jq -r 'select(.rule.id=="5712") | .data.srcip' /var/osse
c/logs/alerts/alerts.json | sort -u
192.168.56.23
analyst@wazuh:~$ sudo jq -r 'select(.data.srcip=="192.168.56.23") | .rule.descr
iption' /var/ossec/logs/alerts/alerts.json | sort | uniq -c | sort -rn
     29 sshd: Attempt to login using a non-existent user
      3 sshd: brute force trying to get access to the system. Non existent use
r.
      1 Host Blocked by firewall-drop Active Response
analyst@wazuh:~$
```

▶ **Practice** Investigate the alerts from your lab test and write a case note (see the triage slide).

FINAL LAB

Detect, respond, report

1. Build the 3-VM lab and install Wazuh (or Splunk).
2. Enroll the web server and confirm its logs arrive.
3. Generate 10 failed SSH logins from Kali. Find rule 5710 / 5712.
4. Add custom rule 100100 and an active response with a 600 s timeout.
5. Edit a file under /etc and find the FIM alert (550).
6. Write a case note: timeline, scope, verdict, tuning.



RECAP

Logs tell the story

A SIEM collects, normalizes and correlates. The tool changes, the method doesn't: good data, clear rules, careful triage and honest case notes.

Next: write detections with Sigma · threat hunting · SOAR automation

